

Data structures, measured

Assignment 2 - In-Memory Workload Engine | Nursultan Malgazhdar | SE-2516 | 2026-10-05

Implementation. DynamicArray and MinHeap store primitive ints in arrays with initial capacity 16, double when full, and retain capacity after removals. MyLinkedList is singly linked with head and tail references; each node stores an int. The two sequences share IntSequence, so identical workloads exercise both.

Complexity of all 13 required operations

Operation	Best	Average / aggregate*	Worst	Aux. worst	Why
Array add(x)	$\Theta(1)$	$\Theta(1)$ amort.	$\Theta(n)$	$\Theta(n)$	Doubling copies n items; copies sum geometrically.
Array add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	Shift $n-i$ cells; possibly allocate a larger array.
Array remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Shift $n-i-1$ cells; capacity is retained.
Array get(i)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	One bounds check and one indexed cell read.
Array contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Stop at first hit or scan all n live cells.
List add(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Tail pointer avoids traversal; allocate one node.
List add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head/tail are direct; otherwise find predecessor.
List remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head is direct; otherwise find predecessor.
List get(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Follow exactly i next links from the head.
List contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Scan nodes until a hit or the null successor.
Heap insert(x)	$\Theta(1)$	$\Omega(1)$, $O(\log n)$ amort. bounds*	$\Theta(n)$	$\Theta(n)$	Bubble up $\log n$ levels; resizing can copy n cells.
Heap peekMin()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	The minimum is at array index 0.
Heap extractMin()	$\Theta(1)$	$\Omega(1)$, $O(\log n)$ *	$\Theta(\log n)$	$\Theta(1)$	Replace root; bubble down; no capacity shrink.

Assumptions. n is the live size before a valid operation; indexed averages use a uniform valid index. Search averages assume a fixed positive miss probability (50% in W2), so the average is $\Theta(n)$ even with duplicates. An immediate match gives the best case; a miss gives the worst case.

***Average is not amortized.** No universal tight average bound is asserted for heap operations without an input distribution. The listed heap bounds are non-tight; insert's $O(\log n)$ is an aggregate amortized upper bound including growth. An individual insertion that resizes costs $\Theta(n)$, even if its bubbling is short. Array append is $\Theta(1)$ amortized, not $\Theta(1)$ worst case.

Space. Auxiliary space excludes retained storage and includes a temporary replacement array during growth. Retained storage is $\Theta(\text{capacity})$ for the array/heap and $\Theta(n)$ for the list; retained capacity can reflect a larger historical size. Bonus buildHeap takes $\Theta(n)$ time in every case and $\Theta(n)$ auxiliary space for its defensive copy.

Why the operations are correct

Two loop invariants and a reproducible measurement contract

1 `DynamicArray.contains(value)`

Invariant. Before iteration i , $0 \leq i \leq \text{size}$ and every live position j with $0 \leq j < i$ has been inspected and differs from `value`. The array and `size` are unchanged.

Initialization. $i = 0$, so the inspected prefix is empty and the statement holds.

Maintenance. `read(i)` returns the current element. Equality returns true with a witnessed match. Otherwise that element also differs; incrementing i extends the inspected prefix by one and preserves the invariant.

Termination. The loop either returns true at a match or ends with $i = \text{size}$. In the latter case every live value differs and false is correct. The nonnegative variant $\text{size} - i$ decreases on each continuing iteration.

Conclusion. Both return paths exactly implement membership; metric increments do not modify stored elements.

2 `MinHeap.bubbleDown(index)`

Precondition and invariant. Let r be the index passed to this call. Its two child subtrees are already heaps. Before every iteration, within the original subtree rooted at r , only edges from the current index to its children may violate heap order; the repaired path above it is ordered and its ancestors remain no larger than values below them. The subtree's multiset is unchanged.

Initialization. After `extractMin`, only the replaced root can violate order. During Floyd's build, previously processed child subtrees are heaps. The path above the initial index is empty; no claim about ancestors outside the subtree at r is required.

Maintenance. The code selects the smaller existing child. If the current value is no larger, it returns with the subtree ordered. Otherwise swapping lifts the smaller child above both child roots and moves the only possible violation into that child's position. Its child subtrees remain heaps, and a swap preserves the multiset.

Termination. Each swap descends one tree level. At a leaf ($\text{index} \geq \text{size}/2$), or at the early return, no violating child edge remains.

Conclusion. The original subtree becomes a min-heap containing the same values. For `extractMin`, the original root was minimum and the remaining heap is restored.

Measurement contract

Data and repetition. Every case recreates new `Random(42)` and the same nonnegative input values. Sizes are 100, 1,000, 10,000 and 100,000. Twenty discarded preheat cycles at $n = 1,000$ precede two per-case warm-ups and five measured runs; `results.csv` stores median wall time and deterministic counters. There are 36 cases and 180 measured runs, also preserved in `raw_runs.csv`. Setup and query generation are excluded from W1-W3; counters reset after filling.

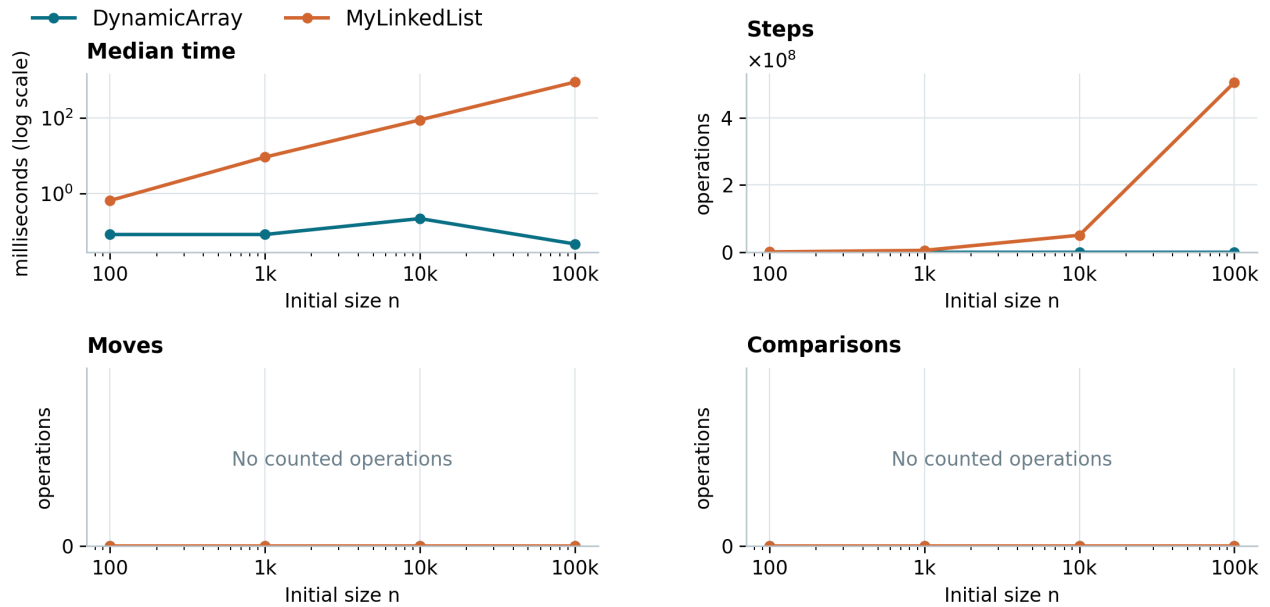
Counting. A step is an array-cell read or a next-reference read (including a successor read for rewiring or null). A move is a write relocating an existing array value or an explicit list head/tail/next update. First placement of an external array value is excluded. Comparisons count element values, not loop bounds. Counters run inside timed operations; these categories are not CPU instruction counts.

Recorded environment. `java.version`: 21.0.12.1; `java.vm.name`: OpenJDK 64-Bit Server VM; `os.name`: Windows 11; `os.arch`: amd64; `available.processors`: 12; `cpu.model`: AMD Ryzen 5 7535HS with Radeon Graphics.

Access and search

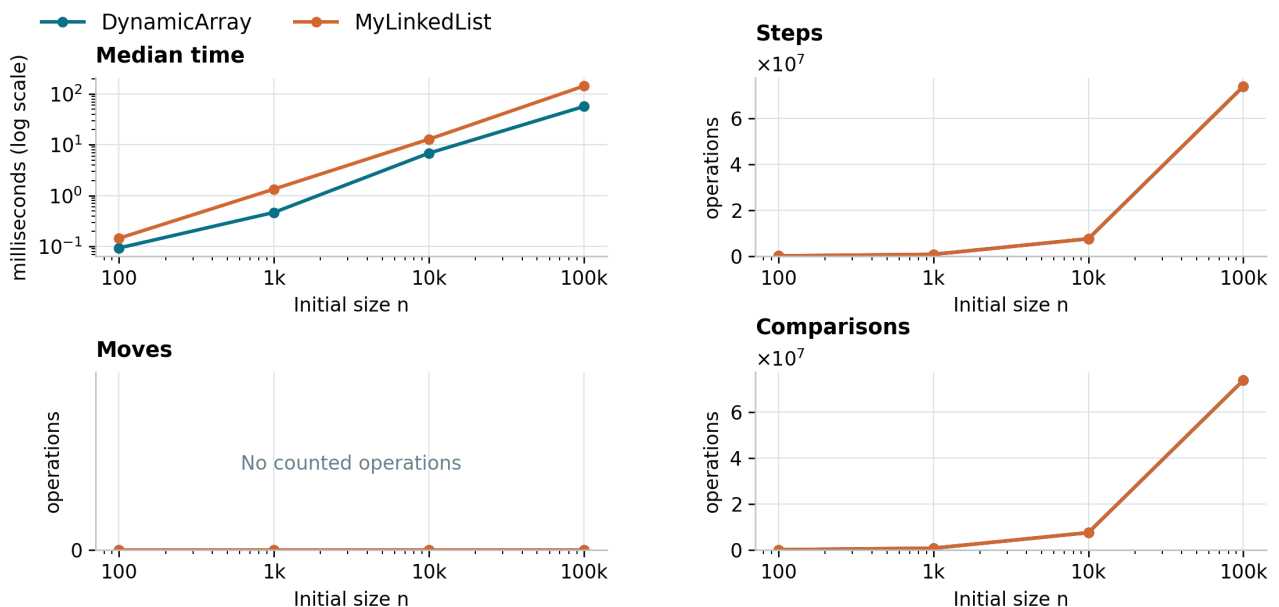
Identical input and query streams expose traversal cost and storage-layout effects

W1 | 10,000 random reads



W1. 10,000 uniformly sampled get(index) calls. At n = 100,000: array 0.046 ms, list 898.768 ms; steps 10,000 versus 504,930,938. Array reads stay constant per query; list traversal grows with the index.

W2 | 1,000 searches: 50% hits

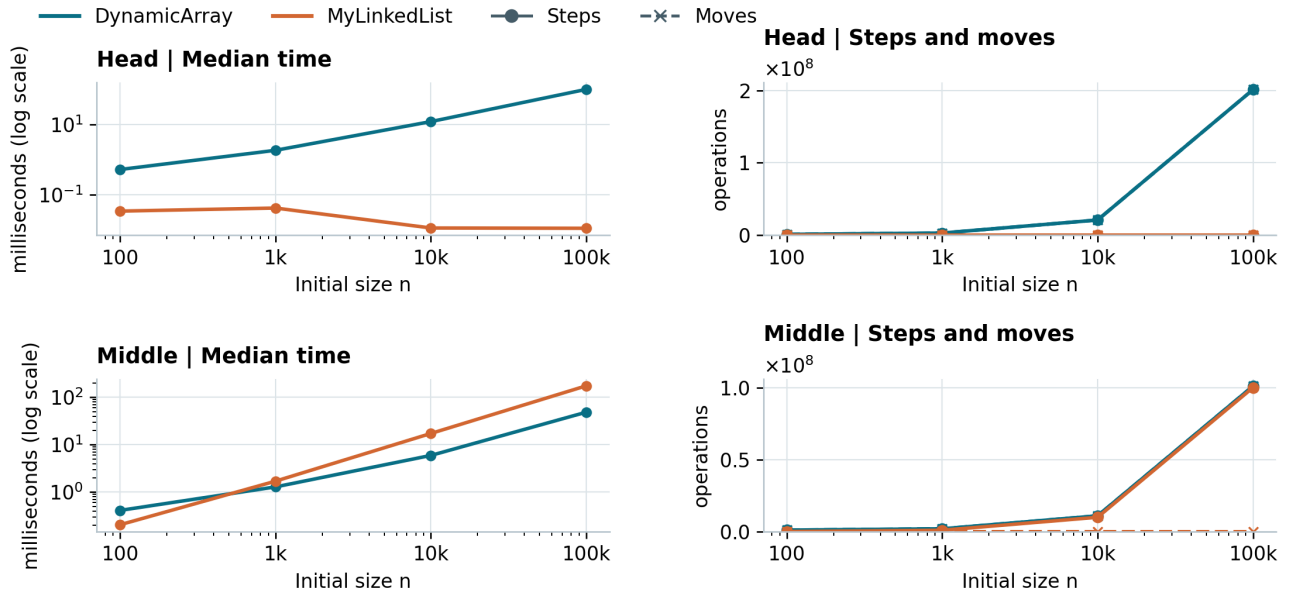


W2. 500 hits sampled from the data and 500 guaranteed negative misses, shuffled before timing. At n = 100,000: array 56.849 ms, list 143.468 ms. Both make 73,835,543 element comparisons; a matching node needs no successor read, so list steps are 500 below its comparisons. Repeated input values can shorten hit scans.

Updates and priority processing

Boundary insertion, indexed traversal, and heap repair are different workloads

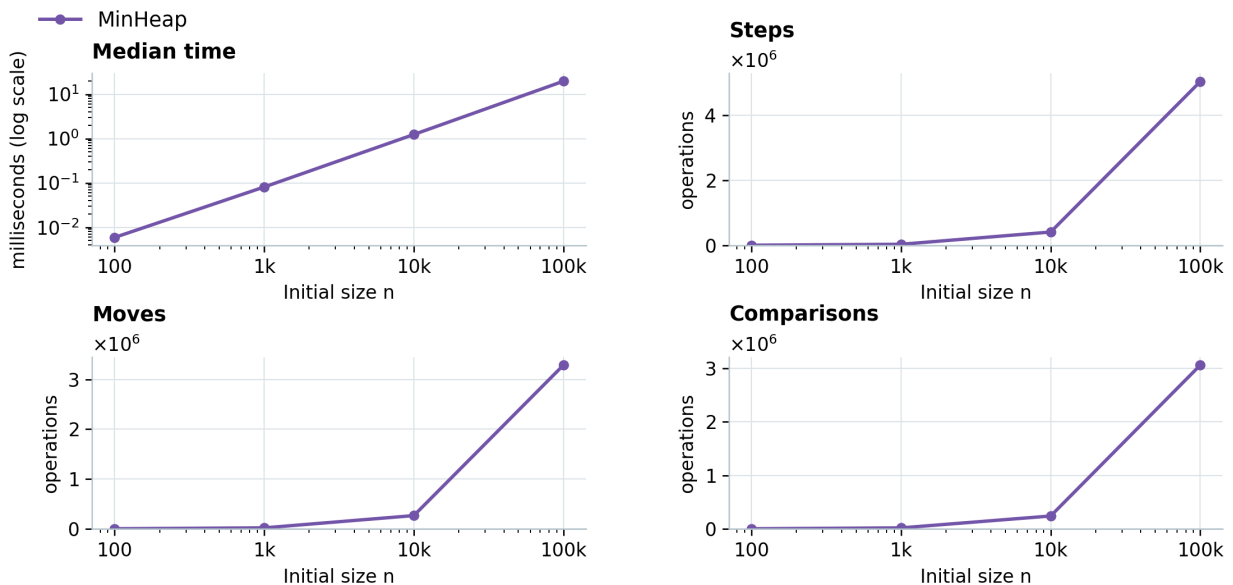
W3 | 1,000 insertions + 1,000 removals



Comparisons = 0 throughout; array steps = moves + 1,000 (curves nearly overlap).

W3. First 1,000 insertions, then 1,000 removals, at index 0 or the original $n/2$ (fixed throughout). At $n = 100,000$, head updates take 102.196 ms for the array and 0.011 ms for the list. The list rewires a constant number of links at the head; the array shifts the live suffix. Middle list updates must find their predecessor.

W4 | n inserts + n minimum extractions



W4. Insert n values into an empty heap, then extract n minima. At $n = 100,000$ this takes 19.660 ms with 3,059,125 element comparisons. The timed loop includes checksum accumulation and non-decreasing-order validation; those validation comparisons are excluded from the structure's counter. The final heap must be empty. Growth copies are included.

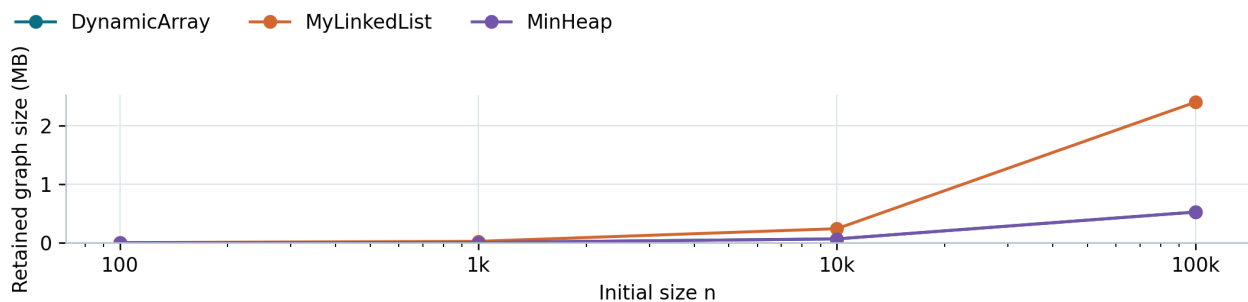
Interpreting the evidence

Twelve discussion sentences, followed by the optional experiments

- 01.** At $n = 100,000$, W1's list/array time ratio is $19,666.69\times$, consistent with 504,930,938 next-link reads versus 10,000 array reads.
- 02.** A contiguous int array packs nearby values into cache lines, so scanning it can benefit from spatial locality and hardware prefetching.
- 03.** W2 performs 73,835,543 comparisons in each structure at $n = 100,000$, yet its list/array time ratio is $2.52\times$.
- 04.** A node traversal depends on the previous pointer load, and separate object headers and links increase memory traffic; equal counted steps therefore need not imply equal elapsed time.
- 05.** The W2 timing is consistent with these layout effects but does not isolate cache misses or garbage-collection pauses, which were not measured separately.
- 06.** For W3 head updates at $n = 100,000$, array/list time is $9,290.51\times$ and list link updates total 3,000, making this list suitable for head edits and tail appends.
- 07.** At the middle index, array and list take 48.633 and 174.951 ms respectively, because an index-based list must locate the predecessor before constant-size rewiring.
- 08.** Removing the last list element still requires traversal in this singly linked design, so a tail reference alone does not make every boundary removal constant time.
- 09.** MinHeap is suitable for repeatedly selecting the next minimum-priority job, with constant-time peeking and logarithmic worst-case extraction rather than arbitrary positional access.
- 10.** Array growth contributes occasional linear spikes, while retained capacity improves subsequent updates at the cost of memory that does not shrink after removals.
- 11.** These measurements include counter increments and use one JVM process with 20 preheat cycles, two per-case warm-ups and five timed repetitions, so tiny cases and scheduler noise still limit cross-machine conclusions.
- 12.** The operation counts support the asymptotic analysis, while median times describe this run and cannot by themselves establish a universal speed ranking.

Bonus experiments

Bonus A | Memory footprint



JOL. At $n = 100,000$ the retained reachable graph is 0.524 MB (array), 2.400 MB (list), and 0.524 MB (heap), including wrapper objects, metrics and spare capacity. A node takes 24 bytes here: 12-byte header + 4-byte int + 4-byte reference + 4-byte alignment padding; the arrays retain 131,072 slots. MB = 1,000,000 bytes; VM layout details are recorded in `jol_vm_details.txt`; Serviceability Agent attachment was unavailable, so no address claims are made.

Floyd. At $n = 100,000$, bottom-up construction uses 188,424 comparisons and 2.308 ms versus 227,662 comparisons and 3.098 ms for repeated insertion. Summing nodes by height gives $\sum n \cdot h/2^{(h+1)} = O(n)$; the input copy supplies $\Omega(n)$. Repeated insertion has $O(n \log n)$ worst-case total time, but random input can cause short upward paths, so a log-factor speedup is not guaranteed.

Sources: DAA Assignment 2 - Data Structures (provided course brief); [OpenJDK Java Object Layout \(JOL\)](#). All plotted observations come from this project's CSV files.